

Teacher-Guided Recovery for GETA-Based Joint Structured Pruning and Quantization: A Comparative Study of Fixed and Compression-Aware Distillation

MD. Sabbir Alam Pial^a, Ayesha Akter Lima^a, Abu Raihan^a, Al Reyad^a

^a*Department of Computer Science and Engineering, Varendra University, Rajshahi, Bangladesh*

Abstract

Model compression is essential for deploying deep neural networks on resource-constrained devices. This paper studies teacher-guided recovery for GETA-based joint structured pruning and weight-only quantization by comparing GETA-only training, fixed vanilla knowledge distillation (KD), and Compression-Aware Knowledge Regularization (CAKR). CAKR activates after target sparsity becomes stable and adapts the teacher signal using compression-state indicators and teacher-correct confidence weighting while keeping cross-entropy as the main objective. Experiments on CIFAR-10 and CIFAR-100 with ResNet20, ResNet56, and VGG7 show that teacher guidance generally improves GETA-compressed models, but the best strategy depends on student capacity and architecture. CAKR improves over GETA-only in all six settings and achieves the highest mean accuracy in three settings; on ResNet20/CIFAR-10, it also reduces the standard deviation across seeds from ± 0.61 to ± 0.13 , indicating more stable recovery. Fixed KD is strongest for higher-capacity students such as ResNet56/CIFAR-100. The results show that adaptive KD is safer for smaller or compression-sensitive students, while fixed KD remains a strong recovery baseline for larger or more stable students.

Keywords: model compression, structured pruning, quantization, knowledge distillation, GETA, CAKR, CNN compression

Email address: `sabbir.vu11@gmail.com` (MD. Sabbir Alam Pial)

1. Introduction

Deep neural networks have achieved strong performance in computer vision tasks, but their computational and memory requirements limit deployment on resource-constrained devices. ResNet56 on CIFAR-100, for example, requires 861K parameters and approximately 125M multiply-accumulate operations per inference — well beyond typical edge device budgets [1, 2]. Model compression is therefore an important step between training and deployment [3, 4]. Among compression techniques, structured pruning and quantization are widely adopted because they reduce both model size and computational cost [5].

Structured pruning removes hardware-friendly components such as channels, filters, or groups, directly reducing model width and multiply-accumulate operations. Quantization reduces numerical precision, lowering storage and inference cost [4, 6, 7, 8, 9, 10, 11]. In practice, these techniques are often applied separately, but this can be suboptimal because pruning changes the network structure and may increase sensitivity to low-precision representation. Joint pruning and quantization addresses this by optimizing sparsity and bit-width under a unified process [12, 13].

GETA provides an automatic framework for joint structured pruning and quantization-aware training, with explicit control over target sparsity and bit-width constraints [13]. Although GETA is a strong compression backbone, the compressed student is primarily optimized with cross-entropy. During aggressive joint compression, the student changes continuously as structured groups are removed and quantization parameters are updated. Teacher guidance through knowledge distillation can help recover lost information, but the best form of teacher guidance is not obvious for this setting.

This paper studies teacher-guided recovery for GETA-based joint compression by comparing three approaches: GETA-only (cross-entropy only), fixed vanilla KD, and CAKR, a compression-aware adaptive regularizer that controls the teacher signal using the current compression state and teacher reliability. Prior work has combined KD with pruning and quantization [14, 15, 16], but none directly addresses fixed versus adaptive KD within GETA-based joint structured pruning and quantization. To the best of our knowledge, few studies have systematically compared GETA-only, fixed KD, and compression-aware KD under the same GETA-based joint pruning and quantization protocol.

The main contributions of this paper are as follows:

- We study teacher-guided recovery for GETA-based joint structured pruning and weight quantization under a common compression protocol.
- We compare GETA-only, GETA with fixed vanilla KD, and GETA with CAKR across ResNet20, ResNet56, and VGG7 on CIFAR-10 and CIFAR-100.
- We propose CAKR as a controlled compression-aware KD variant that uses stable sparsity activation, teacher-correct confidence weighting, and compression-state indicators to adapt the teacher signal during joint compression.
- We show that CAKR achieves the highest mean accuracy in 3 of 6 settings and consistently improves over GETA-only in all settings, while fixed KD is a stronger recovery baseline for higher-capacity students such as ResNet56. Critically, fixed KD reduces accuracy below the GETA-only baseline on ResNet20/CIFAR-10, while CAKR does not.
- We show that CAKR improves compressed-model accuracy stability: on ResNet20/CIFAR-10, the standard deviation across seeds decreases from ± 0.61 (GETA-only) to ± 0.13 (CAKR), suggesting more consistent recovery.

The remainder of this paper is organized as follows. Section 2 reviews related work. Section 3 provides background on GETA and KD. Section 4 describes the compared teacher-guided recovery methods. Section 5 presents the experimental setup. Section 6 reports results and discussion. Section 7 concludes the paper.

2. Related Work

2.1. Structured Pruning and Dependency-Aware Compression

Structured pruning removes channels, filters, or dependency-connected groups to obtain hardware-friendly acceleration [3, 5]. Unlike unstructured pruning, structured pruning directly reduces dense tensor dimensions and multiply-accumulate operations. Channel pruning and filter pruning methods remove components based on importance criteria such as magnitude or activation statistics [5]. DepGraph automates structured pruning by

70 tracing structural dependencies between layers, enabling group-wise pruning across diverse architectures [17]. OTO and OTOv2 further integrate sparsity-inducing optimization directly into training, reducing the need for separate prune-and-fine-tune cycles [18, 19]. These dependency-aware methods motivate the structured group view used by GETA for joint pruning and
 75 quantization.

2.2. Quantization and Quantization-Aware Training

Quantization reduces storage and arithmetic cost by representing weights or activations with fewer bits [4, 6]. Quantization-aware training (QAT) approaches such as DoReFa-Net and PACT enable low-bit training through differentiable approximations of the quantization function [7, 8]. Learned Step
 80 Size Quantization (LSQ) learns quantization step sizes during training and improves low-bit quantization performance [11]. HAWQ and HAWQ-V2 use second-order sensitivity information to assign different bit-widths to different layers, showing that mixed-precision quantization can preserve accuracy
 85 more effectively than uniform quantization [9, 10]. These methods show that bit-width sensitivity is layer-dependent, motivating the learnable quantization approach used in GETA. CAKR is complementary to these quantization methods because it focuses on teacher-guided recovery during GETA-based joint compression rather than changing the quantization scheme.

90 2.3. Joint Pruning and Quantization

Applying pruning and quantization separately can be suboptimal because pruning changes the network structure and may alter quantization sensitivity. Zhang et al. showed that structured channel pruning and uniform quantization should be considered together rather than as independent
 95 post-processing steps [12]. GETA extends this direction by automatically combining structured pruning and quantization-aware training with explicit sparsity and bit-width constraints through a quantization-aware dependency graph and a quantization-aware structured sparse optimizer [13]. In this paper, GETA is used as the common compression backbone for all evaluated
 100 methods.

2.4. Knowledge Distillation for Compressed Models

Knowledge distillation transfers soft output information from a teacher model to a student network [20]. Feature distillation methods such as Fit-Nets, RKD, CRD, and PKT further transfer hidden-layer information and

105 representation geometry [21, 22, 23, 24]. These methods are relevant because CAKR optionally uses a small feature distillation term. Several compression pipelines combine pruning, quantization, and KD to improve recovery: PQK [14] constructs a teacher from pruned weights in a two-phase pipeline, QRPK [15] studies quantization robustness with KD across multiple bit-
 110 widths, and a CAKD-based pipeline [16] combines pruning, QAT, and distillation. PDC proposes a one-stage joint framework for CNN acceleration that reconstructs redundant filters using intra-layer similarity and guides the process with teacher supervision, eliminating separate compression and recovery stages [25]. Malihi and Heidemann investigate the combined effects
 115 of KD and pruning strategies on CIFAR-10 and CIFAR-100 across ResNet, DenseNet, EfficientNet, and MobileNet, finding that the interaction between pruning method and KD affects compression efficiency in an architecture-dependent manner [26]. Cui et al. combine structured pruning and dense knowledge distillation in an end-to-end joint training process for transformer
 120 compression, showing that joint optimization outperforms sequential application of the two techniques [27].

Unlike these methods, CAKR does not require a multi-stage pipeline, an internal teacher, or a fixed distillation approach. It uses the dense pretrained
 checkpoint as the teacher and adapts the KD signal according to the current
 125 compression state during GETA training. Unlike Cui et al. [27], who study transformer language models with a fixed dense KD signal, this work focuses on CNN-based joint structured pruning and weight quantization using GETA and provides a systematic multi-seed comparison of fixed and compression-aware KD strategies within this compression framework. The architecture-
 130 dependent finding of Malihi and Heidemann [26] — that no single pruning-KD pairing is optimal across all architectures — is consistent with our results and motivates the study of adaptive compression-aware KD control.

3. Background

3.1. GETA-Only Compression Objective

135 Let $f_s(x; \theta_s)$ denote the compressed student. In GETA-only training, the student is optimized using cross-entropy while GETA jointly updates pruning and quantization-related variables:

$$\mathcal{L}_{\text{GETA}} = \mathcal{L}_{\text{CE}}(y, f_s(x)). \quad (1)$$

GETA follows a four-stage training process: a *warm-up* stage that initializes all variables using stochastic gradient descent; a *projection* stage that progressively reduces the bit-width range until the bit-width constraint is satisfied; a *joint* stage that simultaneously removes redundant structured groups and updates quantization parameters; and a *cool-down* stage that fine-tunes the remaining compressed parameters. During the joint stage, GETA uses a saliency-based criterion to identify redundant groups and a forget-rate mechanism to progressively zero them out. After training completes, GETA constructs the final compressed subnet by removing the zeroed groups and fixing the learned quantization parameters. In this work, GETA-only denotes this full process using cross-entropy loss only, serving as the primary baseline.

3.2. Vanilla Knowledge Distillation

Let z_t and z_s be teacher and student logits. Vanilla KD minimizes the KL divergence between softened distributions [20]:

$$\mathcal{L}_{\text{KD}} = D_{\text{KL}}\left(\sigma\left(\frac{z_t}{T}\right) \parallel \sigma\left(\frac{z_s}{T}\right)\right), \quad (2)$$

where $\sigma(\cdot)$ denotes the softmax function and T is the temperature. The standard KD objective scales the distillation term by T^2 to preserve gradient magnitude:

$$\mathcal{L} = \mathcal{L}_{\text{CE}} + \lambda_{\text{KD}} T^2 \mathcal{L}_{\text{KD}}. \quad (3)$$

The temperature T controls the softness of the output distributions — higher T produces softer distributions that reveal more inter-class similarity information from the teacher. The T^2 scaling factor preserves the gradient magnitude relative to the cross-entropy term, ensuring that the KD loss contributes meaningfully during training. In vanilla KD, both T and λ_{KD} are fixed throughout training regardless of the student’s current state. Section 4 describes how vanilla KD and the proposed CAKR regularizer are applied on top of this GETA backbone.

4. Teacher-Guided GETA Recovery

4.1. Overview

Two teacher-guided recovery strategies are compared on top of GETA: fixed vanilla KD and CAKR. In all cases, the dense checkpoint is used to initialize the student and, for KD-based methods, as the teacher. GETA

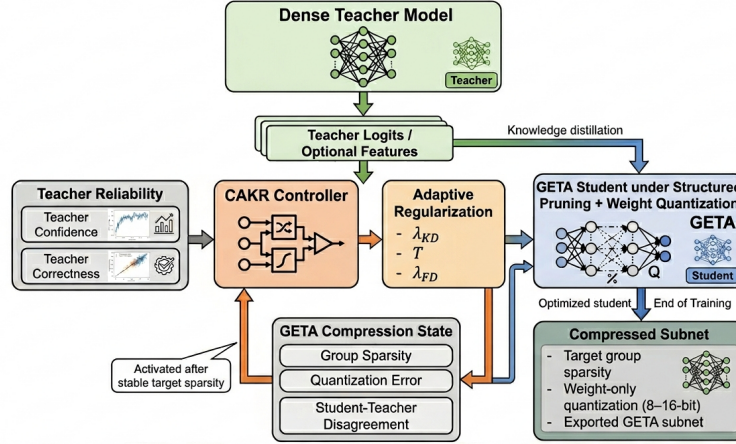


Figure 1: Overview of the GETA+CAKR framework. The dense pretrained model acts as the teacher while the student is compressed using GETA-based structured pruning and weight-only quantization. CAKR adapts KD and optional feature distillation after target sparsity becomes stable.

controls the pruning and quantization process, while the recovery objective controls how teacher information is used during compression. As illustrated in Fig. 1, CAKR activates after the target sparsity becomes stable and adapts the teacher signal using both teacher reliability and compression-state indicators.

4.2. Fixed Vanilla KD Recovery

The fixed KD baseline applies a constant teacher signal after the target sparsity has stabilized for a predefined number of epochs. The loss is Eq. (3) with $T = 3.0$ and a fixed λ_{KD} selected by dataset: 0.06 for CIFAR-10 and 0.08 for CIFAR-100. This baseline tests whether simple teacher guidance is already sufficient for GETA recovery, and serves as an important comparison point rather than a weak ablation.

4.3. Compression-Aware Knowledge Regularization (CAKR)

CAKR keeps cross-entropy as the primary objective and adds adaptive teacher-guided regularization using the teacher-correct weighted KD term defined in Section 4.4:

$$\mathcal{L}_{CAKR} = \mathcal{L}_{CE} + \lambda_{KD} T^2 \mathcal{L}_{KD}^w + \lambda_{FD} \mathcal{L}_{FD}. \quad (4)$$

Here $\mathcal{L}_{\text{CE}}$ is the supervised cross-entropy loss, $\mathcal{L}_{\text{KD}}^w$ is the teacher-correct
 185 confidence-weighted KD loss, $\mathcal{L}_{\text{FD}}$ is the optional feature distillation loss, T
 is the distillation temperature, and λ_{KD} and λ_{FD} control the KD and FD
 contributions. The CE term is not down-weighted. CAKR is activated only
 after the target sparsity is reached and stable, so that teacher guidance does
 not disturb the early GETA pruning and quantization search.

190 4.4. Teacher-Correct Confidence Weighting

To reduce negative transfer, CAKR emphasizes teacher guidance only
 when the teacher is correct and confident. For each training sample i , teacher
 confidence and correctness are defined as:

$$c_i = \max \sigma(z_t^i), \quad r_i = \mathbf{1}[\arg \max(z_t^i) = y_i], \quad w_i = r_i \cdot c_i. \quad (5)$$

The teacher-correct weighted KD loss is:

$$\mathcal{L}_{\text{KD}}^w = \frac{\sum_i w_i D_{\text{KL}}\left(\sigma\left(\frac{z_t^i}{T}\right) \parallel \sigma\left(\frac{z_s^i}{T}\right)\right)}{\sum_i w_i + \varepsilon}. \quad (6)$$

195 This design suppresses KD contributions from incorrect teacher predictions,
 which is particularly important on harder datasets such as CIFAR-100 where
 teacher predictions are less reliable.

4.5. Compression-State Controller

CAKR estimates a compression risk score using the current group sparsity
 200 G , normalized quantization error Q , and teacher-student disagreement D :

$$R = w_1 G + w_2 Q + w_3 D, \quad w_1 = 0.35, \quad w_2 = 0.30, \quad w_3 = 0.10. \quad (7)$$

The KD coefficient and temperature are controlled as:

$$\lambda_{\text{KD}} = \lambda_{\text{KD}}^{\max} \cdot g_{\text{stage}} \cdot g_{\text{conf}} \cdot (1 - R), \quad (8)$$

$$T = T_{\min} + (T_{\max} - T_{\min}) \cdot R. \quad (9)$$

Here $g_{\text{stage}} = 0$ before target sparsity is stable and $g_{\text{stage}} = 1$ afterward;
 $g_{\text{conf}} = \frac{1}{N} \sum_i c_i$ is the batch-averaged teacher confidence. A small feature
 distillation term is optionally added:

$$\mathcal{L}_{\text{FD}} = \sum_l \beta_l \left\| \hat{h}_s^l - \hat{h}_t^l \right\|_2^2, \quad (10)$$

205 where $\hat{h}^l$ denotes normalized features at layer l , with maximum weight $\beta_l = 0.003$. The maximum KD weight $\lambda_{\text{KD}}^{\text{max}}$ is selected per model-dataset pair from $\{0.06, 0.08\}$ and then fixed across all seeds of that setting.

4.6. Training Procedure

210 The training procedure is: (1) train a dense model and save the best checkpoint; (2) initialize the GETA student from the dense checkpoint; (3) use the same dense checkpoint as the teacher for KD-based methods; (4) run GETA compression with CE only during warm-up and early compression stages; (5) compute compression risk R , teacher confidence g_{conf} , and teacher correctness after target sparsity is stable; (6) activate fixed KD or CAKR
215 and optimize the combined objective; and (7) select the best checkpoint after stable target sparsity and export the compressed subnet.

5. Experimental Setup

Experiments are conducted on CIFAR-10 and CIFAR-100 [28] using ResNet20, ResNet56 [29], and VGG7 [30]. CIFAR-10 provides a standard 10-class
220 benchmark, while CIFAR-100 provides a harder 100-class setting that causes larger performance degradation after compression. ResNet20 and ResNet56 evaluate residual networks of different depths; VGG7 is included as a non-residual CNN.

All compressed models use GETA with 60% target group sparsity, weight-
225 only quantization in the 8–16-bit range, 150 compression epochs, batch size 128, SGD with learning rate 0.05, momentum 0.9, weight decay 5×10^{-4} , gradient clipping 1.0, 10 warm-up epochs, and 50 ramp epochs. The best checkpoint is selected only after the target sparsity is reached and stable for 10 epochs. Main results are averaged over seeds 123, 234, and 345, re-
230 ported as mean $\pm$ standard deviation. Since only three seeds are used, results should be interpreted as consistent directional evidence rather than statistically definitive claims.

For vanilla KD, $T = 3.0$ is fixed for all settings, while λ_{KD} is set to 0.06 for CIFAR-10 and 0.08 for CIFAR-100. For CAKR, $\lambda_{\text{KD}}^{\text{max}}$ is selected from
235 $\{0.06, 0.08\}$ per model-dataset pair, with the feature distillation term either disabled or used with maximum weight 0.003. Controller weights are fixed at $w_1 = 0.35$, $w_2 = 0.30$, $w_3 = 0.10$ across all settings.

All experiments were conducted on Kaggle using $2 \times$ NVIDIA Tesla T4 GPUs with PyTorch 1.13.1 and torchvision 0.14.1. Larger-scale experiments

Table 1: Dense baseline metrics used for student initialization and as teacher models.

Model / Dataset	Dense Acc. (%)	Params	MACs
ResNet20 / CIFAR-10	92.96	272.47K	41.00M
ResNet56 / CIFAR-10	94.29	855.77K	125.75M
VGG7 / CIFAR-10	92.62	1.15M	606.00M
ResNet20 / CIFAR-100	69.14	278.32K	40.82M
ResNet56 / CIFAR-100	73.26	861.62K	125.75M
VGG7 / CIFAR-100	71.65	1.17M	152.79M

on Tiny ImageNet, ImageNet, and transformer architectures are left for future work due to compute limitations. Top-1 accuracy is the primary metric. Test loss, MACs, and nonzero parameters are also reported. Table 1 shows the dense baseline metrics. All training logs and per-seed metrics are available at <https://github.com/sabbir211/CAKR>.

6. Results and Discussion

This section compares GETA-only, GETA+vanilla KD, and GETA+CAKR under the same compression protocol. All results are mean $\pm$ standard deviation over three seeds.

Fig. 2 summarizes the accuracy gain over GETA-only for the two teacher-guided variants. CAKR gives a positive gain in every evaluated setting, while fixed vanilla KD is stronger in some higher-capacity or more stable models but falls below GETA-only on ResNet20/CIFAR-10.

Table 2 shows the compression damage and teacher-guided recovery for each setting. CIFAR-100 settings suffer the largest compression damage — ResNet20 loses 18.23 percentage points from its dense baseline under GETA compression alone.

Table 3 reports the main accuracy comparison. Both KD-based methods improve over GETA-only in most settings, but no single strategy is best for all cases. CAKR achieves the highest mean accuracy in 3 of 6 settings. Crucially, vanilla KD reduces accuracy below the GETA-only baseline on ResNet20/CIFAR-10 (87.64% vs. 87.90%), marked with $\dagger$ in Table 2. CAKR achieves 88.05% in the same setting, confirming that compression-aware adaptive control can be beneficial when fixed KD over-regularizes the smaller student.

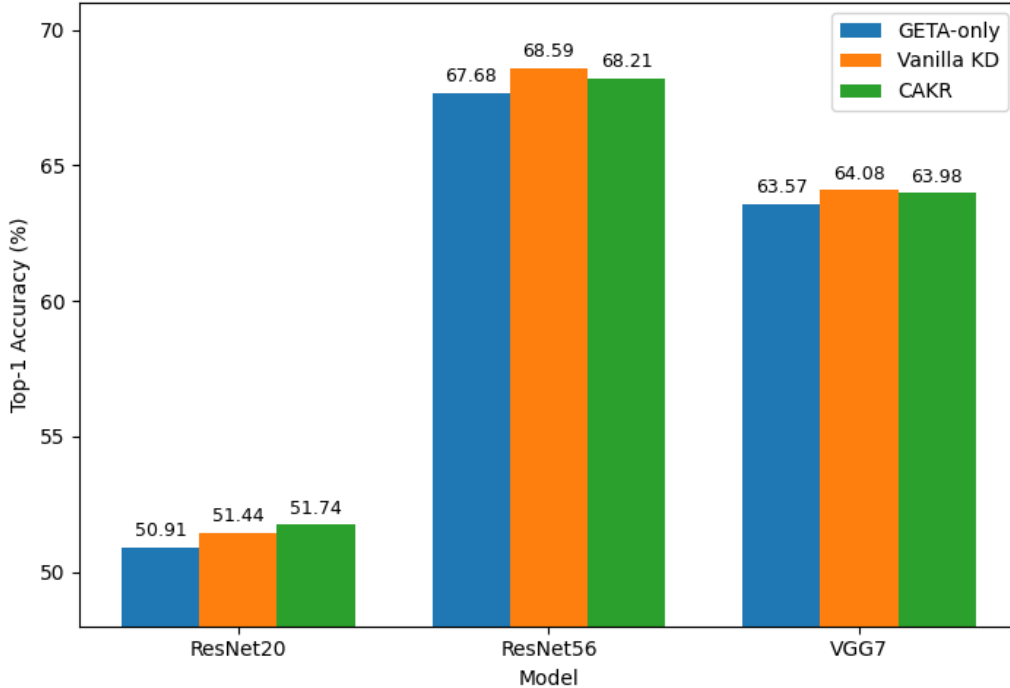


Figure 2: Training dynamics for ResNet20/CIFAR-100 (seed 123). The shaded region indicates the phase after KD activation. CAKR achieves the highest final accuracy with a lower effective distillation coefficient than vanilla KD.

265 6.1. CIFAR-100 Results

Table 4 provides the detailed CIFAR-100 results. Fig. 3 visualizes the top-1 accuracy comparison. CIFAR-100 is the more important benchmark because it has 100 fine-grained classes and causes larger accuracy degradation after compression. Both KD-based methods consistently improve over
270 GETA-only for all three architectures.

On ResNet20/CIFAR-100, CAKR is best in accuracy ($51.74 \pm 0.60\%$) and test loss (1.7585 ± 0.0152). Vanilla KD also improves over GETA-only (51.44%) but is weaker than CAKR. On ResNet56/CIFAR-100, vanilla KD achieves the highest accuracy ($68.59 \pm 0.09\%$), lower loss, and lower MACs,
275 indicating that the deeper higher-capacity student can absorb the stronger fixed signal effectively. CAKR still improves over GETA-only ($+0.53\%$) and produces the most compact subnet (265.77K vs. 267.75K params). On VGG7/CIFAR-100, vanilla KD slightly outperforms CAKR in mean accu-

Table 2: Compression damage and teacher-guided recovery (mean accuracy, %). KD Recovery and CAKR Recovery are gains over the GETA-only baseline. [†]Vanilla KD falls below GETA-only baseline.

Model / Dataset	Dense	GETA-only	KD Recovery	CAKR Recovery
ResNet20 / CIFAR-10	92.96	87.90 (−5.06)	87.64 (−0.26 [†])	88.05 (+0.15)
ResNet56 / CIFAR-10	94.29	93.15 (−1.14)	93.22 (+0.07)	93.26 (+0.11)
VGG7 / CIFAR-10	92.62	90.75 (−1.87)	91.09 (+0.34)	90.88 (+0.13)
ResNet20 / CIFAR-100	69.14	50.91 (−18.23)	51.44 (+0.53)	51.74 (+0.83)
ResNet56 / CIFAR-100	73.26	67.68 (−5.58)	68.59 (+0.91)	68.21 (+0.53)
VGG7 / CIFAR-100	71.65	63.57 (−8.08)	64.08 (+0.51)	63.98 (+0.41)

Table 3: Main accuracy comparison (mean $\pm$ std, 3 seeds). [†]Vanilla KD below GETA-only baseline.

Model / Dataset	GETA-only (%)	Vanilla KD (%)	CAKR (%)	Best
ResNet20 / CIFAR-10	87.90 $\pm$ 0.61	87.64 $\pm$ 0.34 [†]	88.05 $\pm$ 0.13	CAKR
ResNet56 / CIFAR-10	93.15 $\pm$ 0.19	93.22 $\pm$ 0.05	93.26 $\pm$ 0.15	CAKR
VGG7 / CIFAR-10	90.75 $\pm$ 0.22	91.09 $\pm$ 0.13	90.88 $\pm$ 0.20	Vanilla KD
ResNet20 / CIFAR-100	50.91 $\pm$ 0.83	51.44 $\pm$ 0.67	51.74 $\pm$ 0.60	CAKR
ResNet56 / CIFAR-100	67.68 $\pm$ 0.26	68.59 $\pm$ 0.09	68.21 $\pm$ 0.17	Vanilla KD
VGG7 / CIFAR-100	63.57 $\pm$ 0.24	64.08 $\pm$ 0.46	63.98 $\pm$ 0.21	Vanilla KD

280 racy (64.08 $\pm$ 0.46% vs. 63.98 $\pm$ 0.21%), but CAKR has noticeably lower accuracy standard deviation, suggesting more stable recovery.

6.2. CIFAR-10 Results

285 Table 5 shows the CIFAR-10 results. Gains are smaller on CIFAR-10 because GETA-only already achieves high compressed accuracy. The most important finding on CIFAR-10 is that vanilla KD reduces ResNet20 accuracy from 87.90% to 87.64% — falling below the GETA-only baseline. CAKR improves ResNet20 accuracy to 88.05% and reduces the standard deviation from ± 0.61 to ± 0.13 , indicating more stable recovery. For ResNet56/CIFAR-10, CAKR has the highest mean accuracy (93.26%), while vanilla KD has the lowest standard deviation (± 0.05). For VGG7/CIFAR-10, vanilla KD 290 achieves the highest mean accuracy (91.09%), while CAKR (90.88%) still improves over GETA-only.

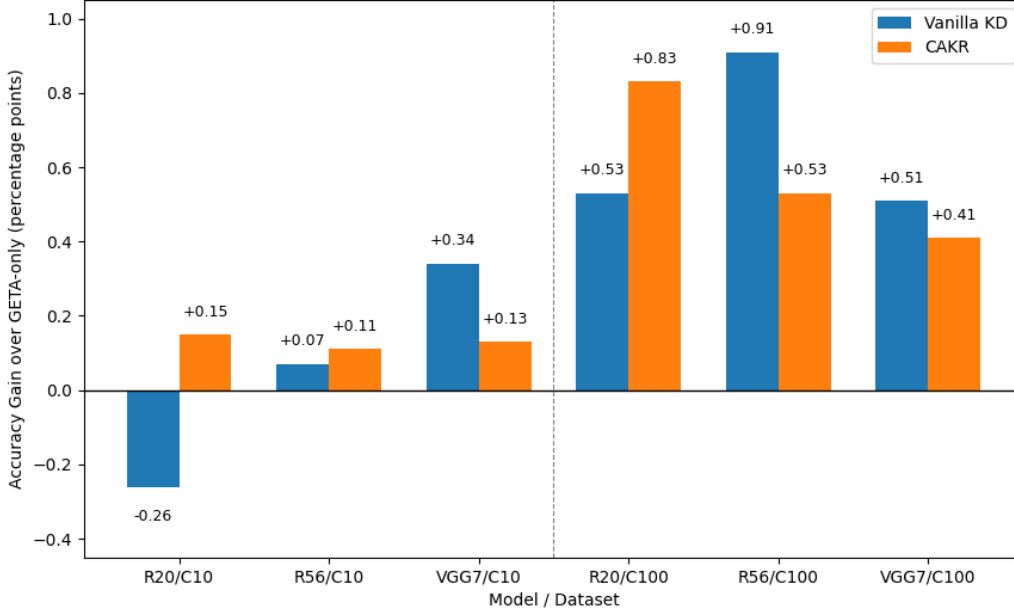


Figure 3: Accuracy comparison across all six settings (mean $\pm$ std, 3 seeds). The annotation marks where vanilla KD falls below GETA-only.

6.3. Effect of Student Capacity and Architecture

The most important finding is that fixed and adaptive teacher guidance are not interchangeable. Vanilla KD uses a fixed effective KD coefficient of 0.54 on CIFAR-10 ($T^2 \times \lambda_{\text{KD}} = 9 \times 0.06$) and 0.72 on CIFAR-100 (9×0.08). Smaller compressed students possess reduced representational capacity and may be unable to absorb a strong constant distillation signal — the student is pulled toward the teacher’s distribution even when doing so conflicts with the compressed model’s own gradient direction. This over-regularization effect is consistent with the ResNet20/CIFAR-10 result, where the fixed signal degrades accuracy below the GETA-only baseline, and with ResNet20/CIFAR-100, where CAKR outperforms vanilla KD despite using a substantially lower effective distillation coefficient. CAKR mitigates over-regularization by scaling the teacher signal according to the current compression state and teacher reliability. In contrast, ResNet56 and VGG7 absorb the stronger fixed signal more effectively, and vanilla KD achieves higher accuracy in three of their four evaluated settings.

CAKR’s adaptive controller reaches a maximum effective coefficient of

Table 4: Detailed CIFAR-100 compression results (mean $\pm$ std, 3 seeds).

Model	Method	Acc. (%)	Test Loss	MACs	Params
ResNet20	GETA-only	50.91 $\pm$ 0.83	1.7849 $\pm$ 0.0371	9.95 $\pm$ 0.45M	44.33 $\pm$ 1.03K
	Vanilla KD	51.44 $\pm$ 0.67	1.7673 $\pm$ 0.0328	9.80 $\pm$ 0.26M	43.79 $\pm$ 1.96K
	CAKR	51.74 $\pm$ 0.60	1.7585 $\pm$ 0.0152	10.23 $\pm$ 0.29M	44.37 $\pm$ 1.91K
ResNet56	GETA-only	67.68 $\pm$ 0.26	1.1641 $\pm$ 0.0193	40.67 $\pm$ 0.76M	267.36 $\pm$ 1.60K
	Vanilla KD	68.59 $\pm$ 0.09	1.1589 $\pm$ 0.0034	39.70 $\pm$ 0.47M	267.75 $\pm$ 2.72K
	CAKR	68.21 $\pm$ 0.17	1.1616 $\pm$ 0.0210	39.80 $\pm$ 1.23M	265.77 $\pm$ 2.51K
VGG7	GETA-only	63.57 $\pm$ 0.24	1.349 $\pm$ 0.004	27.83 $\pm$ 0.49M	157.35 $\pm$ 1.45K
	Vanilla KD	64.08 $\pm$ 0.46	1.290 $\pm$ 0.015	28.84 $\pm$ 0.75M	160.63 $\pm$ 3.31K
	CAKR	63.98 $\pm$ 0.21	1.291 $\pm$ 0.004	29.10 $\pm$ 0.80M	160.06 $\pm$ 2.87K

approximately 0.207–0.257 across settings, substantially lower than the fixed
 310 vanilla KD coefficients of 0.54–0.72, while still consistently improving over
 GETA-only. This supports the interpretation that smaller or more compression-
 sensitive students require controlled teacher guidance, while higher-capacity
 or more stable students can tolerate and benefit from a stronger fixed signal.
 The observed pattern is therefore architecture-dependent.

315 6.4. Limitations of the Evidence

Several limitations should be noted. First, the vanilla KD baseline uses
 only two fixed configurations: $T = 3.0$ with $\lambda_{\text{KD}} = 0.06$ for CIFAR-10
 and $\lambda_{\text{KD}} = 0.08$ for CIFAR-100. More detailed per-model tuning of vanilla
 KD might perform better, particularly on ResNet20/CIFAR-10. However,
 320 CAKR’s compression-state controller naturally adapts the effective KD strength
 without manual per-setting tuning of the KD weight, which may be the prac-
 tical advantage in deployment. Second, results are limited to three seeds
 per setting; larger-scale statistical testing would strengthen the conclusions.
 Third, experiments are restricted to CNNs on CIFAR-10/CIFAR-100; gen-
 325 eralization to larger datasets and transformer architectures remains to be
 validated.

7. Conclusion

This paper studied teacher-guided recovery for GETA-based joint struc-
 tured pruning and weight quantization. Fixed vanilla KD and a compression-

Table 5: Detailed CIFAR-10 compression results (mean $\pm$ std, 3 seeds).

Model	Method	Acc. (%)	Test Loss	MACs	Params
ResNet20	GETA-only	87.90 ± 0.61	0.370 ± 0.016	$14.86 \pm 0.62\text{M}$	40.23K
	Vanilla KD	87.64 ± 0.34	0.407 ± 0.007	$14.05 \pm 0.38\text{M}$	$40.01 \pm 0.66\text{K}$
	CAKR	88.05 ± 0.13	0.390 ± 0.007	$14.44 \pm 0.58\text{M}$	41.38K
ResNet56	GETA-only	93.15 ± 0.19	0.268 ± 0.007	$56.79 \pm 0.51\text{M}$	212.94K
	Vanilla KD	93.22 ± 0.05	0.300 ± 0.002	$56.86 \pm 0.44\text{M}$	$212.46 \pm 1.36\text{K}$
	CAKR	93.26 ± 0.15	0.289 ± 0.013	$56.63 \pm 0.36\text{M}$	213.90K
VGG7	GETA-only	90.75 ± 0.22	0.301 ± 0.001	$41.22 \pm 1.27\text{M}$	$161.01 \pm 3.33\text{K}$
	Vanilla KD	91.09 ± 0.13	0.341 ± 0.007	$42.23 \pm 1.44\text{M}$	$159.70 \pm 1.73\text{K}$
	CAKR	90.88 ± 0.20	0.318 ± 0.006	$41.47 \pm 1.70\text{M}$	$160.92 \pm 0.96\text{K}$

330 aware adaptive variant (CAKR) were compared against GETA-only under the same compression protocol across ResNet20, ResNet56, and VGG7 on CIFAR-10 and CIFAR-100.

Teacher guidance generally improves GETA-compressed models, but the best strategy depends on student capacity and architecture. CAKR achieves 335 the highest mean accuracy in 3 of 6 settings and consistently improves over GETA-only in all settings. A key finding is that fixed KD falls below the GETA-only baseline on ResNet20/CIFAR-10, while CAKR avoids this over-regularization. Fixed KD is stronger for higher-capacity students such as ResNet56/CIFAR-100, where its stronger constant signal is more effectively 340 absorbed. CAKR and fixed KD are therefore complementary strategies: CAKR is safer and more suitable for smaller or more compression-sensitive students, while fixed KD can be more effective for higher-capacity models.

Future work will extend this study to larger datasets such as Tiny ImageNet and ImageNet, transformer architectures supported by GETA, and 345 automated selection of KD strength based on model capacity and compression sensitivity.

Declaration of Competing Interest

The authors declare no competing financial interests.

Data Availability

350 All training logs, per-seed metrics, and figures are publicly available at
<https://github.com/sabbir211/CAKR>.

References

- [1] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, H. Adam, MobileNets: Efficient convolutional neural networks for mobile vision applications, arXiv:1704.04861 (2017).
355
- [2] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, L.-C. Chen, MobileNetV2: Inverted residuals and linear bottlenecks, in: Proc. CVPR, 2018, pp. 4510–4520.
- [3] S. Han, J. Pool, J. Tran, W. J. Dally, Learning both weights and connections for efficient neural network, in: Proc. NeurIPS, 2015, pp. 1135–1143.
360
- [4] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, D. Kalenichenko, Quantization and training of neural networks for efficient integer-arithmetic-only inference, in: Proc. CVPR, 2018, pp. 2704–2713.
365
- [5] Y. He, L. Xiao, Structured pruning for deep convolutional neural networks: A survey, IEEE Trans. Pattern Anal. Mach. Intell. 46 (5) (2024) 2900–2919.
- [6] I. Hubara, M. Courbariaux, D. Soudry, R. El-Yaniv, Y. Bengio, Quantized neural networks: Training neural networks with low precision weights and activations, J. Mach. Learn. Res. 18 (187) (2018) 1–30.
370
- [7] S. Zhou, Y. Wu, Z. Ni, X. Zhou, H. Wen, Y. Zou, DoReFa-Net: Training low bitwidth convolutional neural networks with low bitwidth gradients, arXiv:1606.06160 (2016).
375
- [8] J. Choi, Z. Wang, S. Venkataramani, P. Chuang, V. Srinivasan, K. Gopalakrishnan, PACT: Parameterized clipping activation for quantized neural networks, arXiv:1805.06085 (2018).

- [9] Z. Dong, Z. Yao, A. Gholami, M. Mahoney, K. Keutzer, HAWQ: Hessian aware quantization of neural networks with mixed-precision, in: Proc. ICCV, 2019, pp. 293–302.
- [10] Z. Dong, Z. Yao, D. Arfeen, A. Gholami, M. W. Mahoney, K. Keutzer, HAWQ-V2: Hessian aware trace-weighted quantization of neural networks, in: Proc. NeurIPS, 2020.
- [11] S. K. Esser, J. L. McKinstry, D. Bablani, R. Appuswamy, D. S. Modha, Learned step size quantization, in: Proc. ICLR, 2020.
- [12] X. Zhang, I. Colbert, S. Das, Learning low-precision structured subnetworks using joint layerwise channel pruning and uniform quantization, Appl. Sci. 12 (15) (2022) 7829. doi:10.3390/app12157829.
- [13] X. Qu, D. Aponte, C. Banbury, D. P. Robinson, T. Ding, K. Koishida, I. Zharkov, T. Chen, Automatic joint structured pruning and quantization for efficient neural network training and compression, in: Proc. CVPR, 2025, pp. 15234–15244.
- [14] J. Kim, S. Chang, N. Kwak, PQK: Model compression via pruning, quantization, and knowledge distillation, in: Proc. Interspeech, 2021, pp. 4568–4572. doi:10.21437/Interspeech.2021-248.
- [15] J. Kim, Quantization robust pruning with knowledge distillation, IEEE Access 11 (2023) 26419–26426. doi:10.1109/ACCESS.2023.3257864.
- [16] W. Chae, K. Seo, Pipeline of pruning, knowledge distillation, and quantization for model compression, J. Electr. Eng. Technol. (2025). doi:10.1007/s42835-025-02541-7.
- [17] G. Fang, X. Ma, M. Song, M. B. Mi, X. Wang, DepGraph: Towards any structural pruning, in: Proc. CVPR, 2023, pp. 16091–16101.
- [18] T. Chen, B. Ji, T. Ding, B. Fang, G. Wang, Z. Zhu, L. Liang, Y. Shi, S. Yi, X. Tu, Only train once: A one-shot neural network training and pruning framework, in: Proc. NeurIPS, 2021.
- [19] T. Chen, L. Liang, T. Ding, I. Zharkov, Z. Zhu, OTOv2: Automatic, generic, user-friendly neural network training and compression, in: Proc. ICLR, 2023.

- 410 [20] G. Hinton, O. Vinyals, J. Dean, Distilling the knowledge in a neural network, arXiv:1503.02531 (2015).
- [21] A. Romero, N. Ballas, S. E. Kahou, A. Chassang, C. Gatta, Y. Bengio, FitNets: Hints for thin deep nets, in: Proc. ICLR, 2015.
- [22] W. Park, D. Kim, Y. Lu, M. Cho, Relational knowledge distillation, in: 415 Proc. CVPR, 2019, pp. 3967–3976.
- [23] Y. Tian, D. Krishnan, P. Isola, Contrastive representation distillation, in: Proc. ICLR, 2020.
- [24] N. Passalis, A. Tefas, Learning deep representations with probabilistic knowledge transfer, in: Proc. ECCV Workshops, 2018.
- 420 [25] T. Niu, Y. Teng, L. Jin, P. Zou, Y. Liu, Pruning-and-distillation: One-stage joint compression framework for CNNs via clustering, Image Vis. Comput. 136 (2023) 104738. doi:10.1016/j.imavis.2023.104738.
- [26] L. Malihi, G. Heidemann, Matching the ideal pruning method with knowledge distillation for optimal compression, Appl. Syst. Innov. 7 (4) 425 (2024) 56. doi:10.3390/asi7040056.
- [27] B. Cui, Y. Li, Z. Zhang, Joint structured pruning and dense knowledge distillation for efficient transformer model compression, Neurocomputing 458 (2021) 56–69. doi:10.1016/j.neucom.2021.05.084.
- [28] A. Krizhevsky, Learning multiple layers of features from tiny images, 430 Tech. rep., University of Toronto (2009).
- [29] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: Proc. CVPR, 2016, pp. 770–778.
- [30] K. Simonyan, A. Zisserman, Very deep convolutional networks for large-scale image recognition, in: Proc. ICLR, 2015.